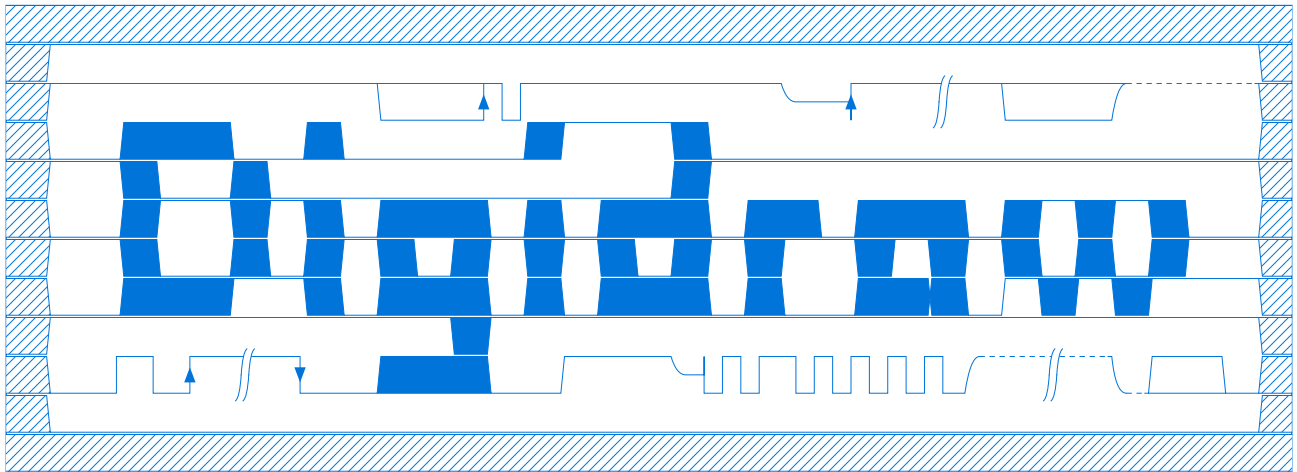




Created by Joel von Rotz

Source Code on [Codeberg](#)

DIGIDRAW is a Typst package to help with drawing digital timing diagrams. The syntax tries to be like from [WaveDrom](#), although is slightly different in some areas. In addition, the package supports more customization such as configuring colors, fonts and others.

The package uses **CETZ** to draw everything.

⚠ Caution

Not all symbol combinations have been implemented so far (probably) or they differ from WaveDrom, so expect some undefined or unexpected behaviour.

If they are straying too far away from the timing diagram standard, please report them in the package repository: <https://codeberg.org/joelvonrotz/typst-digidraw/>

Contents

I \	Installation	4
I.1	From Universe (aka. Stable Version)	4
I.2	From Repository (aka. Development Version)	4
II \	Usage	4
II.1	Drawing Diagrams	4
II.2	Debugging Diagrams	5
II.3	Data Structure	6
II.3.a	Data-Format: Dictionaries	6
II.3.b	Data-Format: JSON	7
II.3.c	Structure	7

III \ Symbols	8
III.1 Clocks	8
III.1.a Normal Clocks p P	8
III.1.b Inverted Clocks n N	8
III.2 Wires	8
III.2.a Straight Signals l L h H	8
III.2.b Flanked Signals 0 1	8
III.2.c High-Impedance Signal z	8
III.2.d Transition Edges u d	9
III.3 Buses	9
III.3.a Don't Care x	9
III.3.b Simple Bus = 2	9
III.3.c Coloured Buses 3 4 5 6 7 8 9	10
III.4 Extender .	10
III.5 Time Skip 	10
IV \ API Reference	11
IV.1 subwave	12
IV.1.a Parameters	12
pos	12
..wave-args	12
IV.2 wave	13
IV.2.a Parameters	13
data	13
debug	14
symbol-width	15
symbol-height	15
step1	16
step2	16
step3	16
bezier-controlpoint	17
edge-overshoot	17
wave-gutter	18
name-gutter	18
s-spacing	19
s-width	19
s-outside	19
mark-scale	20
stroke	20
stroke-dashed	21
show-ticks	21
tick-gutter	22
tick-overshoot	22
tick-format	23
tick-stroke	23
guide-stroke	24
name-format	24
data-format	25
bus-colors	26
others	27

tick-layer	28
wave-layer	28
name	28
IV.3 digidraw-x-pattern	28
IV.3.a Parameters	28
stroke	28
size	29
dir	29
V \ Tips & Tricks	30
V.1 Pre-Configuring the #wave Function	30
V.2 Making Diagrams Referenceable	30
V.3 Diagram and Wave Anchors	31
VI \ Symbol Matrix	33
VII \ Changelog	34
Version 9.0 – <i>Initial release of digidraw</i>	34
Version 9.1	34
Version 9.2	34
Version 9.3	34
Highlight	34
Added	34
Changed	34
Removed	35
Version 9.4	35
Highlight	35
Added	35
Changed	35
Credits	36
Disclaimer	36

I \ Installation

I.1 From Universe (aka. Stable Version)

To import the Universe version, insert following snippet

```
#import "@preview/digidraw:0.9.4" as dd
```

I.2 From Repository (aka. Development Version)

The repository will always be the *development* version of [DIGIDRAW](#). This will also include the latest changes, but might break here and there.

- I. Clone the Repository into your project (for example into a `./packages/` folder or at root level).

```
cd /path/to/project/
git clone https://codeberg.org/joelvonrotz/typst-digidraw.git
```

OR if your project is a git project, you can add it as a submodule, saving a little bit of space in your repository:

```
cd /path/to/project/
git submodule add https://codeberg.org/joelvonrotz/typst-digidraw.git
```

- II. Include the package into your document

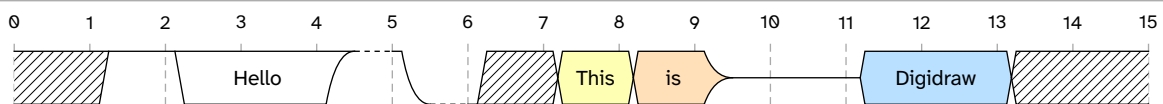
```
#import "./path/to/typst-digidraw/lib.typ" as dd
```

II \ Usage

II.1 Drawing Diagrams

Drawing a diagram is easy:

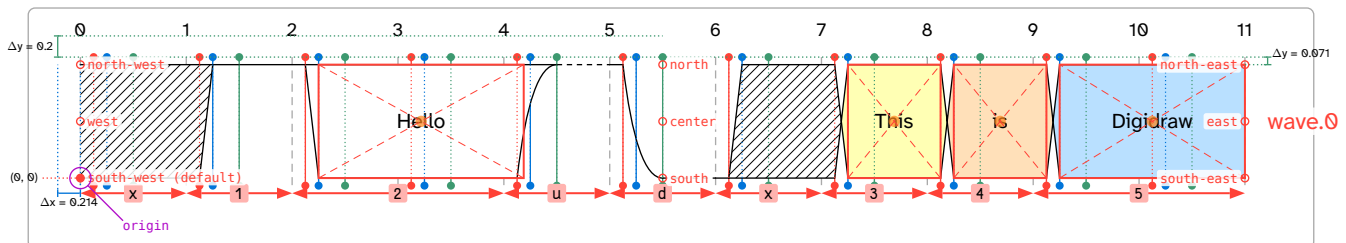
```
#dd.wave(
  (signal: (
    (wave: "x12.udx34z.5.x.", data: "Hello This is Digidraw"),)
  )
)
```



II.2 Debugging Diagrams

You can turn on Debug mode by setting `debug: true` in the `wave()` function. This shows where and which symbol is placed and how long said symbol is. Additionally the `step1`, `step2` and `step3` parameters are shown with respective coloured vertical lines. Debug mode also shows the exact placement of bus labels using a bounding box  and a center point .

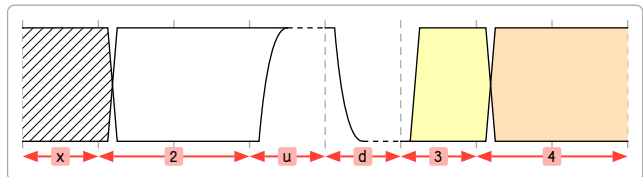
```
#dd.wave(
  (signal: (
    (wave: "x12.udx345.", data: "Hello This is Digidraw"),
  )),
  debug: true
)
```



This looks quite cluttered and is not the actual recommended way to use this feature. Instead you can pass one or more of "symbols", "labels", "coordinates", "steps" and "anchors" to show the respective information.

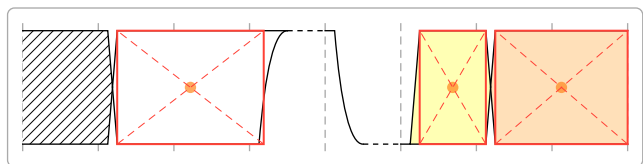
- "symbols" – shows the arrows below the respective wave to show which shape belongs to which symbol.

```
#dd.wave(
  (signal: ((wave: "x2.ud34.")),),
  tick-format: none,
  debug: "symbols"
)
```



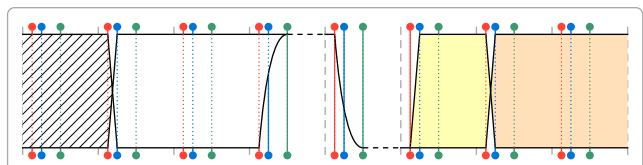
- "labels" – shows the area in which labels can be placed. By default labels are placed horizon+center in the respective box. This can be changed per entry by enclosing the label in a `#align(..)`.

```
#dd.wave(
  (signal: ((wave: "x2.ud34.")),),
  tick-format: none,
  debug: "labels"
)
```



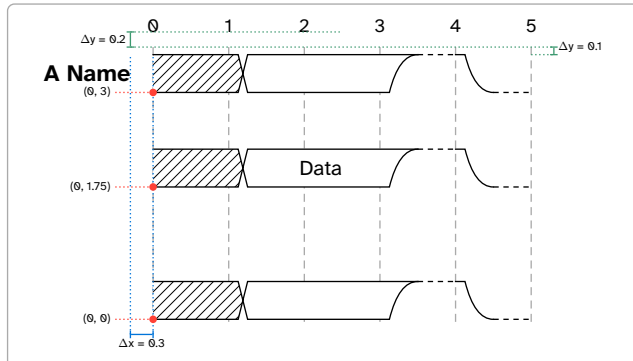
- "steps" – shows the «steps» of each symbol. These are the x coordinates (per symbol) used for transition (i.e. 0→1) and bezier curves (i.e. 2→u).

```
#dd.wave(
  tick-format: none,
  (signal: ((wave: "x2.ud34.")),),
  debug: "steps"
)
```



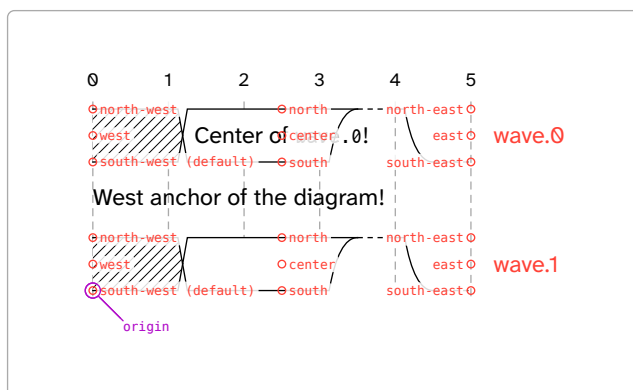
- "coordinates" – shows various (and hopefully useful) coordinates when rendering cetz elements inside or around the timing diagram. The highlighted lower left corner of each wave is **the default anchor point**, when referencing a wave without any directions (i.e. "wave.0"). See the next debug type for more info.

```
#dd.wave(
  symbol-width: 1cm,
  symbol-height: 5mm,
  wave-gutter: 7.5mm,
  (signal: (
    (wave: "x2.ud", name: "A Name"),
    (wave: "x2.ud", data: ([Data],)),
    (:),
    (wave: "x2.ud"))
  ),
  debug: "coordinates"
)
```



- **"anchors"** – shows the various anchor points of each wave. **Additionally**, the whole diagram *itself* has anchor points for all directions ("center", "north", "north-east", "east", "south-east", "south", "south-west", "west" and "north-west").

```
#dd.wave(
  wave-gutter: 1cm,
  (signal: (
    (wave: "x2.ud"),
    (wave: "x2.ud"),
  )),
  others: (info) => {
    import dd.cetz.draw: *
    content("wave.west", anchor: "west", [West anchor of
the diagram!])
    content("wave.0.center", [Center of `wave.0`!])
  },
  debug: "anchors"
)
```



II.3 Data Structure

Let's talk about wave data structures! The data syntax is the one from WaveDrom. You can copy *WaveDrom* scripts and insert those into your document. Well, almost... there are certain features such as groups that are not yet implemented and are thus ignored.

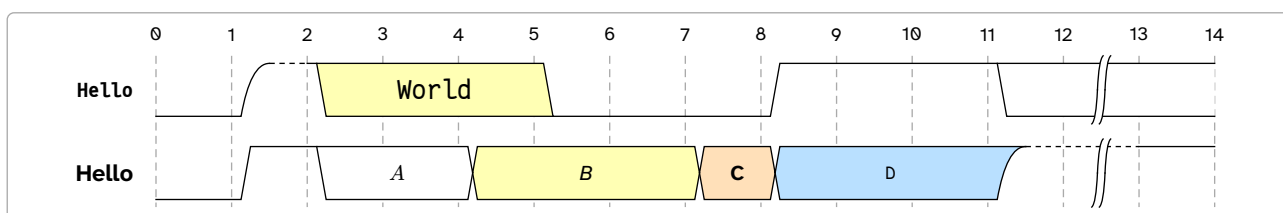
There's two ways on how to define the wave data: as dictionaries or as JSON structures

II.3.a Data-Format: Dictionaries

This allows for fancier styling labels. `.wave` and `.data` can contain *content*. Raw'em, Strong'em, Emph'em, do what you want, as long as it turns into a content.

```
#let data = (signal: (
  (wave: "0u3..0..1..2|.", name: `Hello`, data: (text(1.5em, `World`))),
  (wave: "012.3..45..u|1", name: "Hello", data: ($A$, [_B_], [*C*], `D`))
))

#dd.wave(data)
```

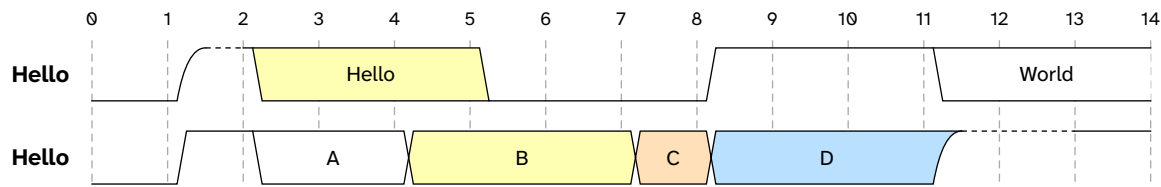


II.3.b Data-Format: JSON

Easier when copying from WaveDrom with the downside of not having much styling options compared to the dictionary based structure.

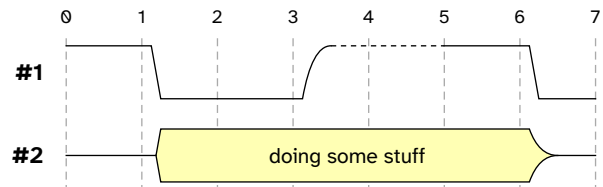
```
#let data = json(bytes(`{"signal": [
  {"wave": "0u3..0..1..2..", "name": "Hello", "data": ["Hello","World"]},
  {"wave": "012.3..45..u.1", "name": "Hello", "data": "A B C D"}
]}`.text)) // or json(read("file.json"))

#dd.wave(data)
```



II.3.c Structure

```
#let data = (
  signal: (
    ( wave: "10.u.10", name: "#1" ),
    (
      wave: "z3...z",
      name: "#2",
      data: ([doing some stuff],)
    ),
  ),
)
#dd.wave(data)
```



.signal array of dictionary

A list/array containing the wave objects.

.wave string

The wave's contents built using the [Symbols](#).

Example: "10..u.10"

.name string or content

The name of the wave.

Example: "Mike" or [Mike]

.data string or array of content or array of string
 JSON, Typst Typst only JSON, Typst

When buses are defined in the wave, labels can be inserted. The index of the label defines its position: first entry goes into the first bus, second in the second one, etc. If data is a string, labels are split by spaces.

i.e. "Hello World" is equal to ("Hello", "World")

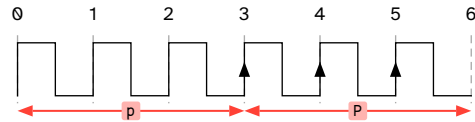
III \ Symbols

III.1 Clocks

III.1.a Normal Clocks p P

Capitalizing the letters adds an arrow mark to the line pointing up.

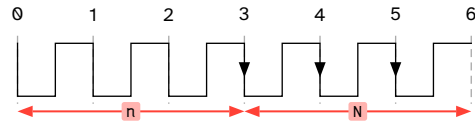
```
#dd.wave(
  (signal: ((wave: "pppPP"),)),
  debug: "symbols"
)
```



III.1.b Inverted Clocks n N

Capitalizing the letters adds an arrow mark to the line pointing up.

```
#dd.wave(
  (signal: ((wave: "nnnNN"),)),
  debug: "symbols"
)
```

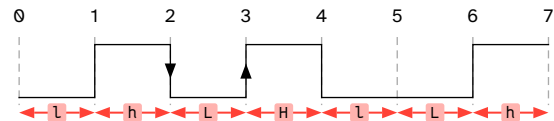


III.2 Wires

III.2.a Straight Signals l L h H

Capitalizing the letters adds an arrow mark to the line pointing up.

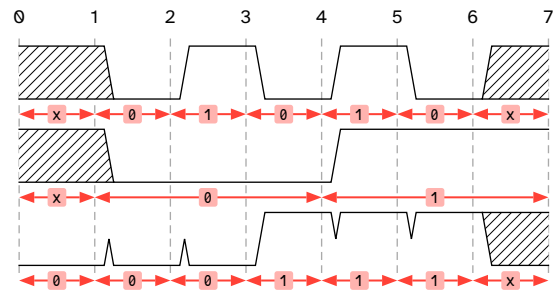
```
#dd.wave(
  (signal: ((wave: "lhLHLh"),)),
  debug: "symbols"
)
```



III.2.b Flanked Signals 0 1

Using 0 and 1 instead of l/L and h/H inserts flanked/sloped signals.

```
#dd.wave(
  (signal: (
    (wave: "x01010x"),
    (wave: "x0..1.."),
    (wave: "000111x"),
  )),
  debug: "symbols"
)
```



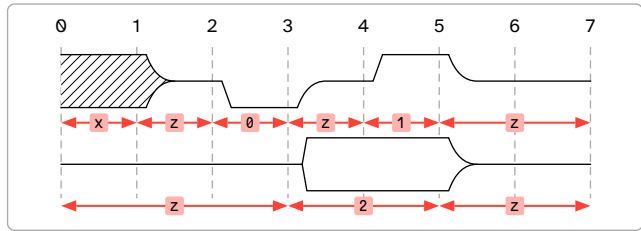
III.2.c High-Impedance Signal z

The high impedance signal introduces curved transitions from any signal to high impedance.

Note multiples of the symbols in a sequence are treated as one long one

Examples "zzz2.zz" → "z..2.z."


```
#dd.wave(
  (signal: (
    (wave: "xz0z1z."),),
    (wave: "zzz2.zz"),),
  ),
  debug: "symbols"
)
```



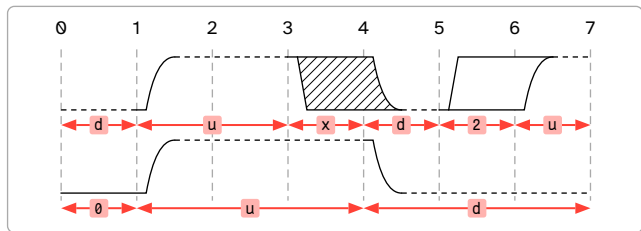
III.2.d Transition Edges u d

If the transition has of unknown duration/delay, a transition edge can be used.

Note multiples of the symbols in a sequence are treated as one long one

Examples "uuuddduuu" → "u..d..u.."

```
#dd.wave(
  (signal: (
    (wave: "du.xd2u"),),
    (wave: "0uuudd"),),
  ),
  debug: "symbols"
)
```



III.3 Buses

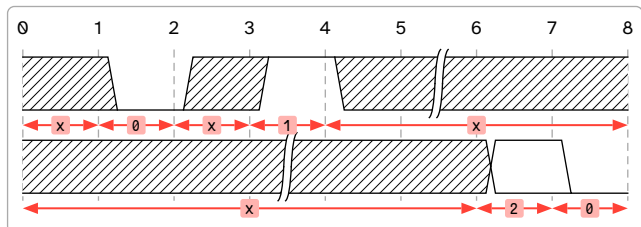
III.3.a Don't Care x

Indicates a section, where the signal value is not important, aka. *don't care*.

Note multiples of the symbols in a sequence are treated as one long one

Examples "xxx|xxx20" → "x..|...20"

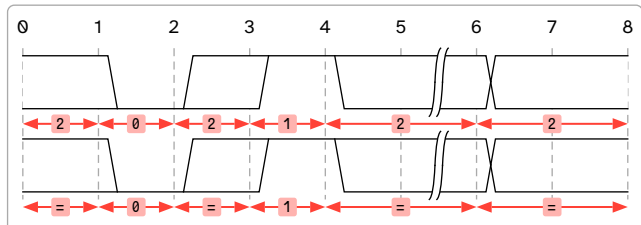
```
#dd.wave(
  (signal: (
    (wave: "x0x1x|x."),),
    (wave: "xxx|xx20"),),
  ),
  debug: "symbols"
)
```



III.3.b Simple Bus = 2

= and 2 are similar in that they have the same bus color. That's it, nothing special, really!

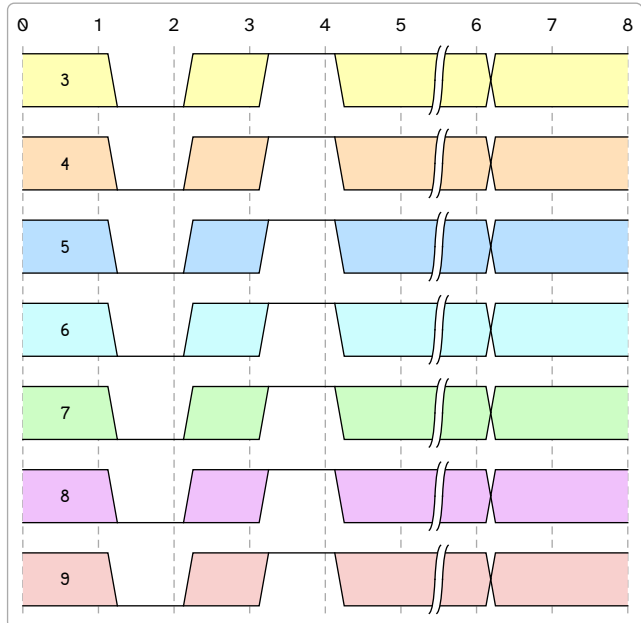
```
#dd.wave(
  (signal: (
    (wave: "20212|2."),),
    (wave: "=0=1|=."),),
  ),
  debug: "symbols"
)
```



III.3.c Coloured Buses 3 4 5 6 7 8 9

The databus can be also colored using numbers "3" through "9".

```
#dd.wave(
  (signal: (
    (wave: "30313|3.", data: (^3^,)),
    (wave: "40414|4.", data: (^4^,)),
    (wave: "50515|5.", data: (^5^,)),
    (wave: "60616|6.", data: (^6^,)),
    (wave: "70717|7.", data: (^7^,)),
    (wave: "80818|8.", data: (^8^,)),
    (wave: "90919|9.", data: (^9^,)),
  )),
)
```

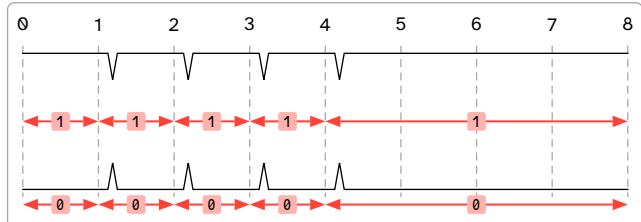


III.4 Extender .

The extender can be used to extend signals. If a signal doesn't look right, you might have to replace some of the symbols with the extender. And if that doesn't work, report it!

For example, while a "1111" inserts high signals with impulses, "1..." stretches.

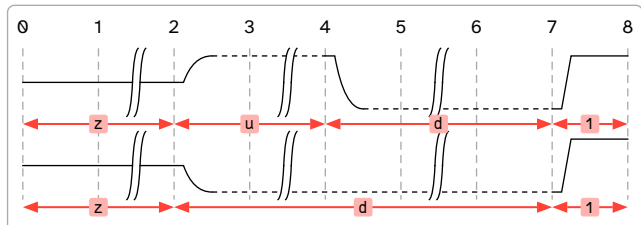
```
#dd.wave(
  (signal: (
    (wave: "11111..."),
    (wave: "00000..."),
  )),
  debug: "symbols"
)
```



III.5 Time Skip |

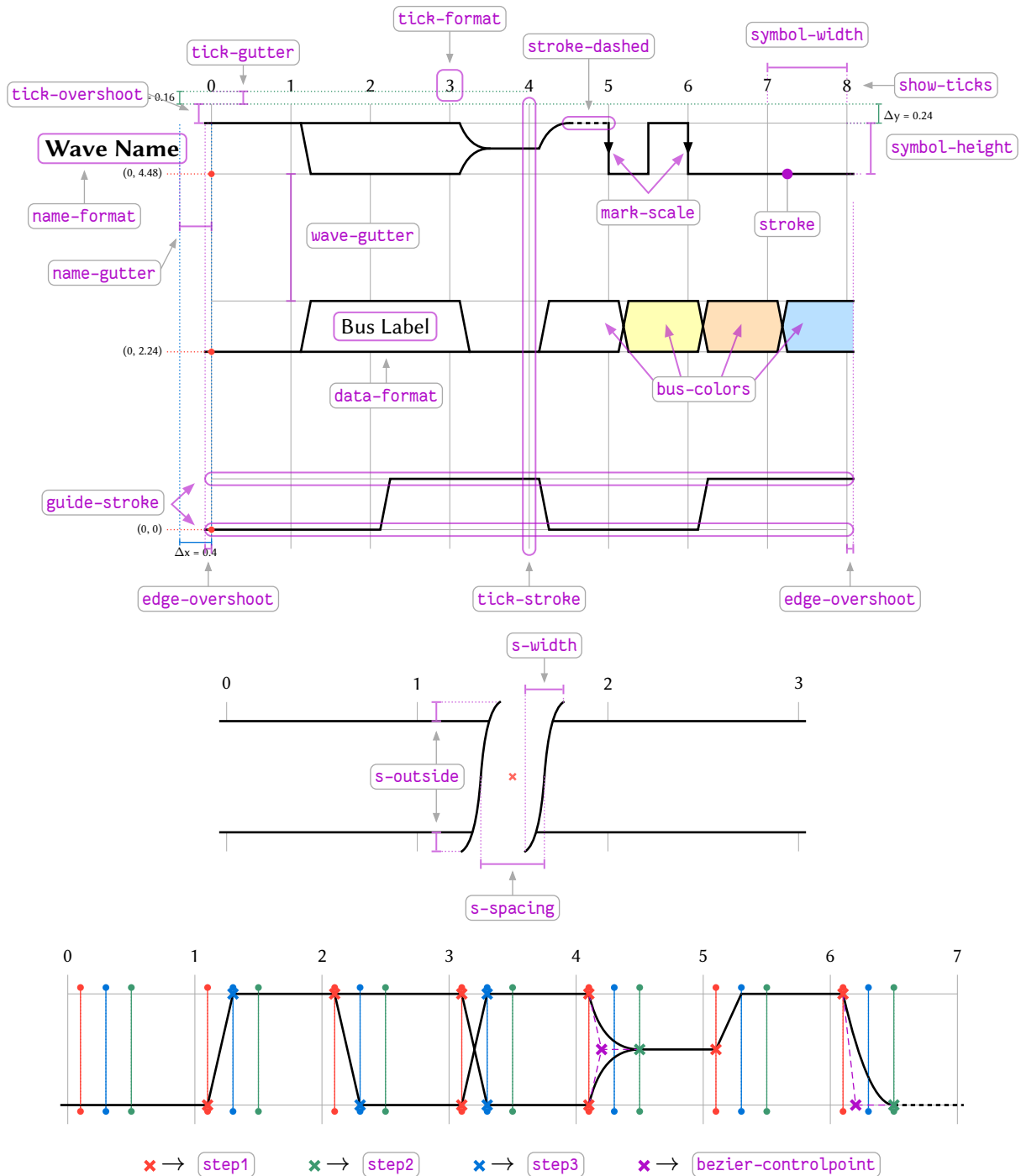
To insert a timeskip to indicate the passing of a long time, insert the symbol "|".

```
#dd.wave(
  (signal: (
    (wave: "z|u|d|.1"),
    (wave: "z|d|d|.1"),
  )),
  debug: "symbols"
)
```



IV \ API Reference

A lot of things in a timing diagram can be customized, ranging from sizing and gutter spacing to coloring and text formatting. Below the most important and useful parameters are shown and their graphical effect.



Note values assigned to `step1`, `step2`, `step3` and `bezier-controlpoint` are X-axis components. The Y-axis components are assigned by the `#wave` function. The example above has the values 10%, 30%, 50%, 20% respectively.

IV.1 subwave

A small wrapper, that returns `cetz` elements instead of the finished `cetz.canvas` object. Through the use of the `no-canvas` parameter, the return value of `wave` returns a tuple of (y-scale, bodies).

But you don't need to know all the specifics. Use the function for example in a `cetz` canvas.

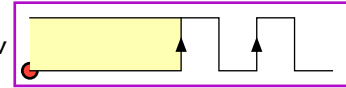
```
#import "@preview/cetz:0.5.2"

#let data = (signal: ((wave: "3.P.", name: [Hello
Digidraw])),)

#cetz.canvas({
  import cetz.draw: *
  circle((0,0), radius: 0.1, fill: red, name: "mike")
  dd.subwave("mike", data, name: "peter")

  rect(
    (rel: (-0.2,-0.2), to: "peter.0.south-west"),
    (rel: (0.2,0.2), to: "peter.0.north-east"),
    stroke: purple
  )
})
```

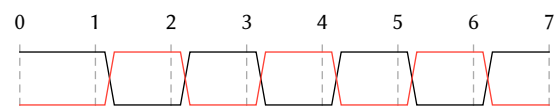
Hello Digidraw



or inside `wave`. Note that the default name of either `wave` and `subwave` is "wave":

```
#let data = (signal: ((wave: "1010101")),)
#let subdata = (signal: ((wave: "0101010")),)

#dd.wave(
  data,
  others: (info) => {
    dd.subwave("wave.0.south-west",
      subdata,
      stroke: red
    )
  }
)
```



The helper function differs slightly from `wave` in following things:

- The first parameter is a `cetz` position value (tuple or anchor). Use debug: "anchors" to see the anchors for each wave.
- Tick numbers and lines are not shown per default, but can be activated by setting the respective parameter.
- The function returns `cetz` elements instead of the completed diagram, so you can also place it directly into a `cetz` canvas.

IV.1.a Parameters

```
subwave(
  pos: str, array,
  ..wave-args: arguments
) → cetz
```

pos str, array

`cetz` position value. Can be either a string or an anchor. Check out Section [V](#) for more!

..wave-args arguments

The same arguments as of `wave`

IV.2 wave

The one and only

IV.2.a Parameters

```

wave(
  data: dictionary,
  debug: string array bool,
  symbol-width: length,
  symbol-height: length ratio float,
  step1: ratio float length,
  step2: ratio float length,
  step3: ratio float length,
  bezier-controlpoint: ratio float length,
  edge-overshoot: ratio float length,
  wave-gutter: ratio float length,
  name-gutter: ratio float length,
  s-spacing: ratio float length,
  s-width: ratio float length,
  s-outside: ratio float length,
  mark-scale: float,
  stroke: stroke function,
  stroke-dashed: stroke,
  show-ticks: bool function,
  tick-gutter: ratio float length,
  tick-overshoot: ratio float length,
  tick-format: none function,
  tick-stroke: none stroke function,
  guide-stroke: none stroke dictionary,
  name-format: function,
  data-format: function,
  bus-colors: dictionary,
  others: cetz,
  tick-layer: int,
  wave-layer: int,
  name: string,
  no-canvas
)

```

data dictionary

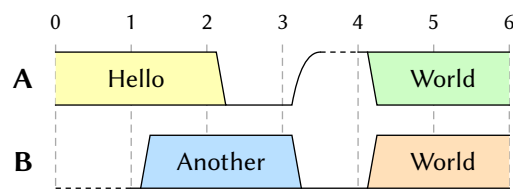
The wave data à la WaveDrom syntax. Do note that the dictionary structure must be like in the example (name and data are optional).

```

#let data = (signal: (
  (wave: "3.0u7.", name: "A", data: "Hello World"),
  (wave: "d5.04.", name: "B", data: ([Another],
[World]),)),
))

#dd.wave(data)

```



debug `string` or `array` or `bool`

Passing one (as a string) or more (as an array) of `"symbols"`, `"steps"`, `"coordinates"`, `"labels"` and `"anchors"`, displays the respective debugging information. Passing `true` enables all informations at once (**not recommended**). `false` disables it (default).

"symbols" Displays where each symbol is located and their length

"steps" Displays `step1`, `step2` and `step3` via vertical lines (as these parameters are applied to every symbol)

"coordinates" Shows coordinates (in cetz units), which comes in handy when placing elements via `others` into the diagram.

"labels" Renders a bounding box and a center point of where a bus label/data can be placed. Using `align(..)` defines the location (see default of `data-format` on how it is used)

"anchors" Shows all the possible anchors, where you can attach elements to. The right side

```
#dd.wave(debug: "symbols",
(signal: ((wave: "1.30.1"),)))

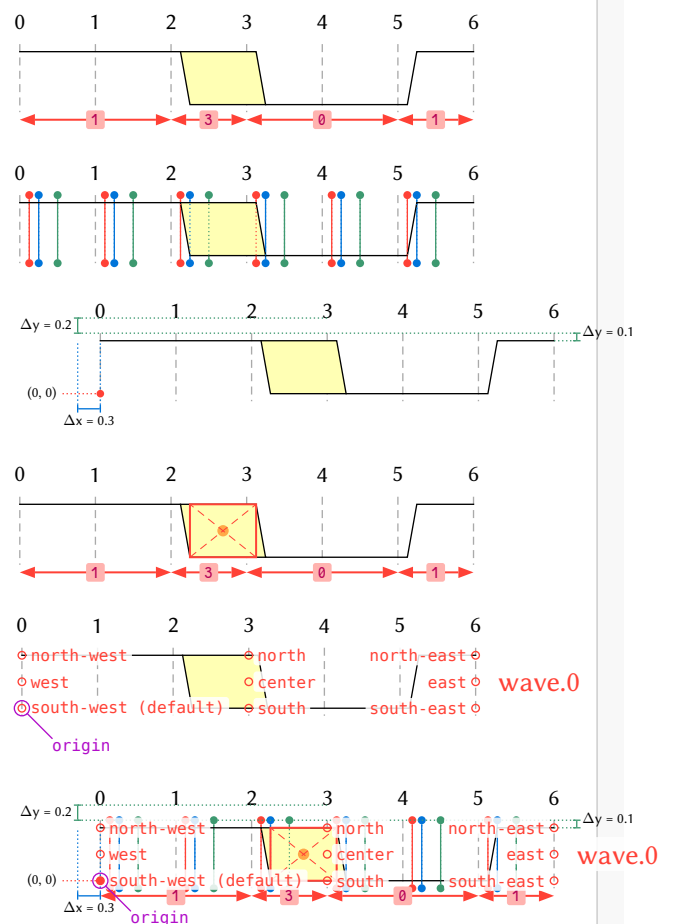
#dd.wave(debug: "steps",
(signal: ((wave: "1.30.1"),)))

#dd.wave(debug: "coordinates",
(signal: ((wave: "1.30.1"),)))

#dd.wave(debug: ("labels", "symbols"),
(signal: ((wave: "1.30.1"),)))

#dd.wave(debug: "anchors",
(signal: ((wave: "1.30.1"),)))

// not recommended:
#dd.wave(debug: true,
(signal: ((wave: "1.30.1"),)))
```



Default: `false`

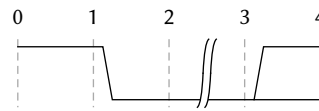
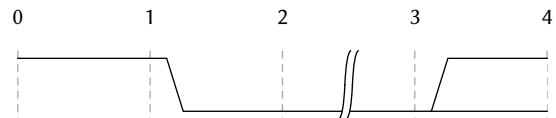
symbol-width length

The width of one symbol and also the reference size of the diagram. All other length-based parameters are scaled to cetz units using this parameter ($l_{\text{relative}} = l_{\text{absolute}} / \text{symbol-width}$)

```
#let data = (signal: ((wave: "10|2"),))

*symbol-width: 1cm* (default)
#dd.wave(data)

*symbol-width: 1.75cm*
#dd.wave(data, symbol-width: 1.75cm)
```

symbol-width: 1cm (default)**symbol-width: 1.75cm**

Default: **1cm**

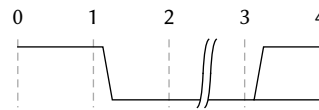
symbol-height length or ratio or float

The height of one symbol or one wave, but applied to all waves. Float and ratio values are treated as relative to symbol-width.

```
#let data = (signal: ((wave: "10|2"),))

*symbol-height: 7mm* (default)
#dd.wave(data)

*symbol-height: 50%*
#dd.wave(data, symbol-height: 50%)
```

symbol-height: 7mm (default)**symbol-height: 50%**

Default: **7mm**

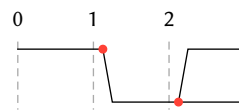
step1 ratio or float or length

First inset at which a transition change starts (i.e. "0" → "1"). The value is the X-axis component, while the Y-axis component is handled by `wave`. The example below highlights the inset with red circles (not actually drawn in real-use).

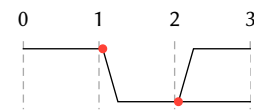
```
#let data = (signal: ((wave: "102"),))

#stack(dir: ltr, spacing: 5mm, [
  *12.5%* (default)
  #dd.wave(data)
], [
  *0.5mm*
  #dd.wave(data, step1: 0.5mm)
])
```

12.5% (default)



0.5mm



Default: 12.5%

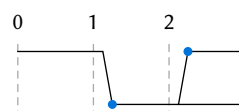
step2 ratio or float or length

Second inset at which a transition change ends (i.e. "0" → "1"). The value is the X-axis component, while the Y-axis component is handled by `wave`. The example below highlights the inset with blue circles (not actually drawn in real-use).

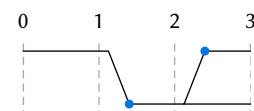
```
#let data = (signal: ((wave: "102"),))

#stack(dir: ltr, spacing: 5mm, [
  *25%* (default)
  #dd.wave(data)
], [
  *4mm*
  #dd.wave(data, step2: 4mm)
])
```

25% (default)



4mm



Default: 25%

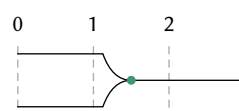
step3 ratio or float or length

Third inset at which a tristate-like transition ends (i.e. "1" → "z"). The value is the X-axis component, while the Y-axis component is handled by `wave`. The example below highlights the inset with green circles (not actually drawn in real-use).

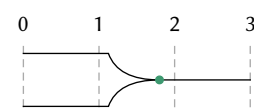
```
#let data = (signal: ((wave: "=z."),))

#stack(dir: ltr, spacing: 5mm, [
  *50%* (default)
  #dd.wave(data)
], [
  *8mm*
  #dd.wave(data, step3: 8mm)
])
```

50% (default)



8mm



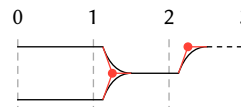
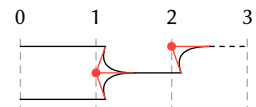
Default: 50%

bezier-controlpoint ratio or float or length

Controls the points used for the bezier curves for curved transitions ("u", "d" and "z"). The value is the X-axis component, while the Y-axis component is handled by `wave`. The example below highlights the bezier control point as red circles and its start and end points (controlled via `step1` and `step3` respectively) with red lines (both are not actually drawn in real-use).

```
#let data = (signal: ((wave: "=zu"),))

#stack(dir: ltr, spacing: 5mm, [
  *25%* (default)
  #dd.wave(data)
], [
  *0%*
  #dd.wave(data, bezier-controlpoint: 0%)
])
```

25% (default)**0%**

Default: **25%**

edge-overshoot ratio or float or length

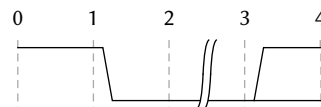
The overshoot on the left and right side of each wave.

```
#let data = (signal: ((wave: "10|2"),))

*edge-overshoot: 0%* (default)
#dd.wave(data)

*edge-overshoot: 5mm*
#dd.wave(data, edge-overshoot: 5mm)

// 100% is equal to 7mm (symbol-width = 7mm)
```

edge-overshoot: 0% (default)**edge-overshoot: 5mm**

Default: **0%**

wave-gutter ratio or float or length

The spacing between waves. Not included when an empty wave is inserted (wave = (:)).

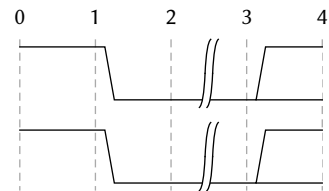
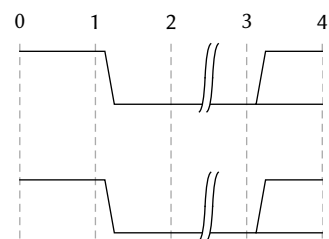
Float and ratio values are treated as relative to symbol-width.

```
#let data = (signal: ((wave: "10|2"),(wave: "10|2")),)
```

```
*wave-gutter: 4mm* (default)
#dd.wave(data)
```

```
*wave-gutter: 100%*
#dd.wave(data, wave-gutter: 100%)
```

```
// 100% is equal to 7mm (symbol-width = 7mm)
```

wave-gutter: 4mm (default)**wave-gutter: 100%**

Default: 4mm

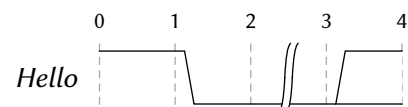
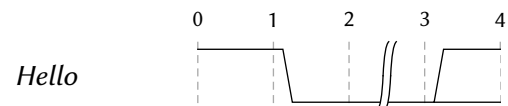
name-gutter ratio or float or length

The spacing between wave and the wave name.

```
#let data = (signal: (
  (wave: "10|2", name: emph[Hello]),
))
```

```
*name-gutter: 0.4* (default)
#dd.wave(data)
```

```
*name-gutter: 1.6*
#dd.wave(data, name-gutter: 1.6)
```

name-gutter: 0.4 (default)**name-gutter: 1.6**

Default: 3mm

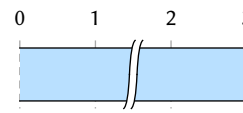
s-spacing ratio or float or length

Spacing between the $\int$ symbols of the time skip symbol.

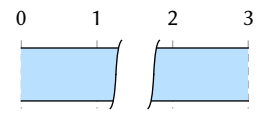
```
#let data = (signal: ((wave: "5|."),))

#stack(dir: ltr,[
  *s-spacing: 1mm*
  #dd.wave(data)

  (default)
],[
  *s-spacing: 50%*
  #dd.wave(data, s-spacing: 50%)
], spacing: 5mm)
```

s-spacing: 1mm

(default)

s-spacing: 50%

Default: 1mm

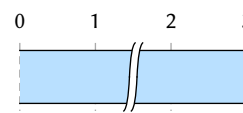
s-width ratio or float or length

Width of the $\int$ symbols of the time skip symbol.

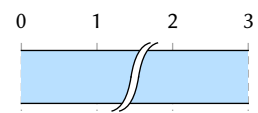
```
#let data = (signal: ((wave: "5|."),))

#stack(dir: ltr,[
  *s-width: 1.5mm*
  #dd.wave(data)

  (default)
],[
  *s-width: 50%*
  #dd.wave(data, s-width: 50%)
], spacing: 5mm)
```

s-width: 1.5mm

(default)

s-width: 50%

Default: 1.5mm

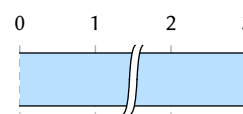
s-outside ratio or float or length

The «overshoot» of the $\int$ symbols of the time skip symbol outside of the wave height.

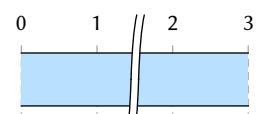
```
#let data = (signal: ((wave: "5|."),))

#stack(dir: ltr,[
  *s-outside: 1mm*
  #dd.wave(data)

  (default)
],[
  *s-outside: 50%*
  #dd.wave(data, s-outside: 50%)
], spacing: 5mm)
```

s-outside: 1mm

(default)

s-outside: 50%

Default: 1mm

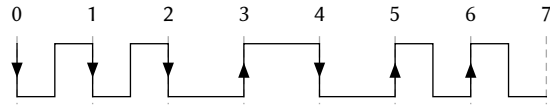
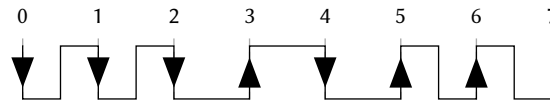
mark-scale float

The size of the arrow marks applied to clocks ("N", "P") and logic signal flanks ("L", "H").

```
#let data = (signal: (
  (wave: "NNLHLP"),
))

*mark-scale: 1* (default)
#dd.wave(data)

*mark-scale: 2*
#dd.wave(data, mark-scale: 2)
```

mark-scale: 1 (default)**mark-scale: 2**

Default: 1

stroke stroke or function

The styling of the solid wave strokes. Aside a stroke, a function in the shape of (n) => {..} -> stroke can be passed to style waves based on their inserted indexes (top starts with index 0).

```
#let data = (signal: ((wave: "10|uz3x"),))

*Default*
#dd.wave(data)

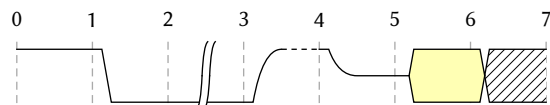
#raw("(paint: red, join: \"round\", cap: \"round\")",
lang: "typc")

#dd.wave(data, stroke: (paint: red, join: "round",
cap: "round"))

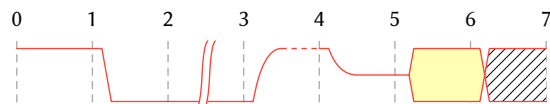
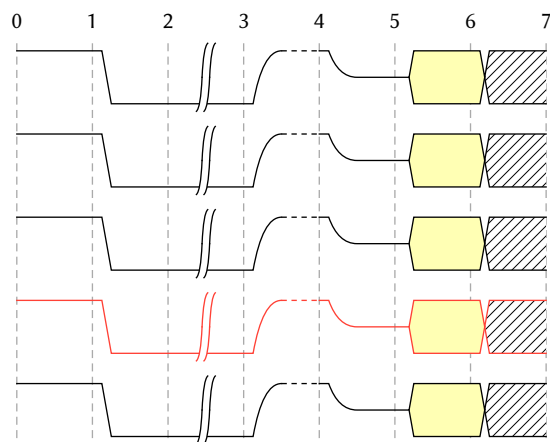
*Functions*
#let style(n) = {
  if n = 3 {red}
  else {black}
}

#let data = (signal: (
  (wave: "10|uz3x"),
  (wave: "10|uz3x"),
  (wave: "10|uz3x"),
  (wave: "10|uz3x"),
  (wave: "10|uz3x"),
))

#dd.wave(data, stroke: style)
```

Default

(paint: red, join: "round", cap: "round")

**Functions**

Default: `stroke(thickness: 0.5pt, cap: "square", paint: black)`

stroke-dashed stroke

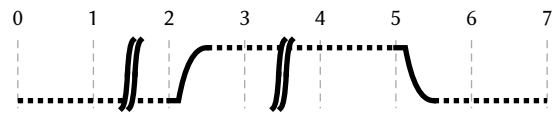
The styling of the dashed strokes of the symbols "u" and "d". Missing configurations are inherited from stroke (as seen in the example below). If for example no color is specified, stroke's color will be used.

```
#let data = (signal: ((wave: "d|u|.d."),))

*Inherited from `stroke`*
#dd.wave(data, stroke: 2pt)

#raw("(paint: red, dash: \"dotted\")", lang: "type")

#dd.wave(data, stroke: 1pt, stroke-dashed: (paint:
red, dash: "dotted"))
```

Inherited from stroke

(paint: red, dash: "dotted")



Default: stroke(dash: (2pt, 1.75pt), cap: "butt")

show-ticks bool or function

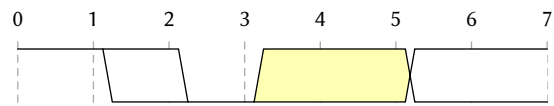
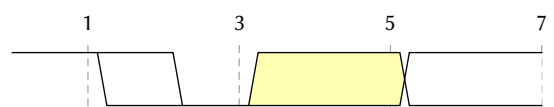
Controls the drawing of the tick lines and labels based on boolean conditions. true draws all tick lines and labels, false does not. Passing a function in the shape of $(n) \Rightarrow \{. .\} \rightarrow \text{bool}$ can be used to conditionally draw tick lines (e.g. only odd lines).

```
#let data = (signal: ((wave: "1203.2."),))

*show-ticks: `true`* (Default)
#dd.wave(data)

*show-ticks: `false`*
#dd.wave(data, show-ticks: false)

*show-ticks: `calc.odd`*
#dd.wave(data, show-ticks: (n) => calc.odd(n))
```

show-ticks: true (Default)**show-ticks: false****show-ticks: calc.odd**

Default: true

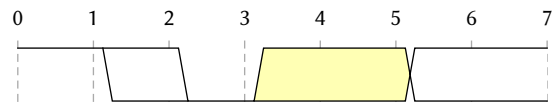
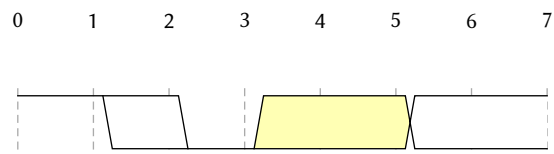
tick-gutter ratio or float or length

Spacing between the tick numbers and tick lines (including tick-overshoot). Length is relative to symbol-width.

```
#let data = (signal: ((wave: "1203.2."),))

*tick-gutter: `20%`* (Default)
#dd.wave(data)

*tick-gutter: `8mm`*
#dd.wave(data, tick-gutter: 8mm)
```

tick-gutter: 20% (Default)**tick-gutter: 8mm**

Default: **20%**

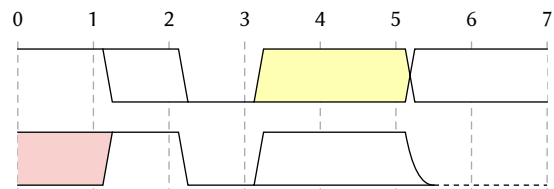
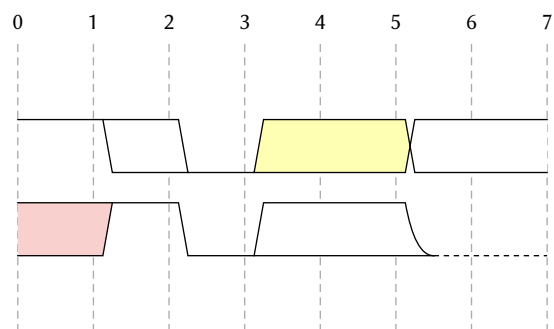
tick-overshoot ratio or float or length

Overshoot of the tick line outside of the **total** diagram height. Length is relative to symbol-width.

```
#let data = (signal: ((wave: "1203.2."), (wave:
"9102.d.")))

*tick-overshoot: `1mm`* (Default)
#dd.wave(data)

*tick-overshoot: `100%`*
#dd.wave(data, tick-overshoot: 100%)
```

tick-overshoot: 1mm (Default)**tick-overshoot: 100%**

Default: **1mm**

tick-format `none` or `function`

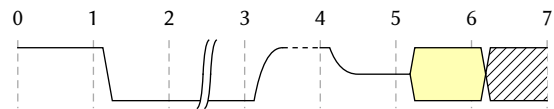
A rendering function with one parameter representing the tick number. Must return content and essentially describes how the label numbers are rendered. Setting it to `none` removes the labels.

```
#let data = (signal: (
  (wave: "10|uz3x"),
))

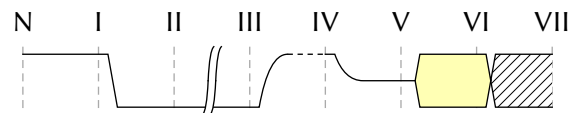
*Default*
#dd.wave(data)

*tick-format:* `(n) => numbering("I", n)`
#dd.wave(data, tick-format: (n) => numbering("I", n))

*tick-format:* `none`
#dd.wave(data, tick-format: none)
```

Default

tick-format: (n) $\Rightarrow$ `numbering("I", n)`



tick-format: `none`



Default: `n` $\Rightarrow$ `text(0.8em, numbering("I", n))`

tick-stroke `none` or `stroke` or `function`

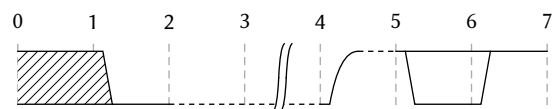
The stroke of the tick lines. `none` disables the lines and passing a function in the shape of (n) $\Rightarrow$ {..} $\rightarrow$ `stroke` | `none` allows for special styling based on tick number.

```
#let data = (signal: ((wave: "x0d|u=1"),))

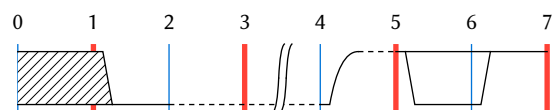
*Default*
#dd.wave(data)

*tick-stroke:* `function`
#dd.wave(data, tick-stroke: (n) => {
  if calc.odd(n) {
    red + 2pt
  } else {
    blue + 0.5pt
  }
})

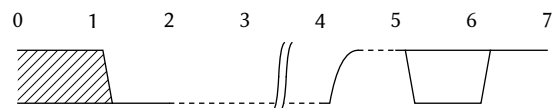
*tick-stroke:* `none`
#dd.wave(data, tick-stroke: none)
```

Default

tick-stroke: `function`



tick-stroke: `none`



Default: (thickness: `0.5pt`, paint: `gray`, dash: `"densely-dashed"`)

guide-stroke `none` or `stroke` or `dictionary`

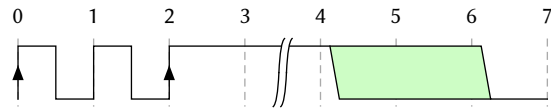
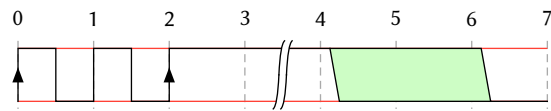
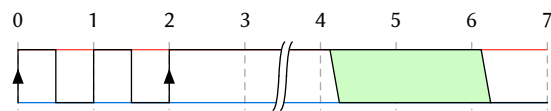
When set to a stroke or a dictionary with stroke entries top and bottom, this will draw horizontal lines at the *Low* and *High* level of each wave.

```
#let data = (signal: ((wave: "PpH|7.0"),))

*guide-stroke: `none`* (Default)
#dd.wave(data)

*guide-stroke: `stroke`*
#dd.wave(data, guide-stroke: red + 0.5pt)

*guide-stroke: `dictionary`*
#dd.wave(data, guide-stroke: (top: red + 0.5pt,
bottom: blue + 0.5pt))
```

guide-stroke: none (Default)**guide-stroke: stroke****guide-stroke: dictionary**

Default: `none`

name-format `function`

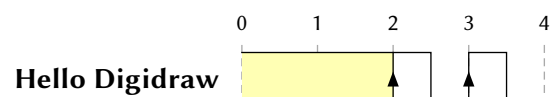
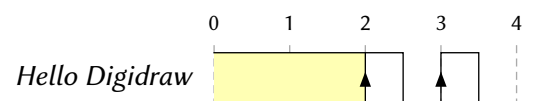
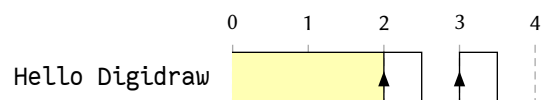
The formatting of the wave names, which are attached next to each respective wave.

```
#let data = (signal: ((wave: "3.P.", name: [Hello
Digidraw])),)

*Default*
#dd.wave(data)

*name-format: `emph`*
#dd.wave(data, name-format: emph)

*name-format: `raw`* (name is `content` here)
#dd.wave(data, name-format: (name) => raw(name.text))
```

Default**name-format: emph****name-format: raw** (name is content here)

Default: `name` => `text(1em, weight: "bold", bottom-edge: "baseline", name)`

data-format function

The formatting of the wave bus data/labels. Similar to `name-format`, with the exception that `data-format` also can use `#align` to align the labels in the respective bus.

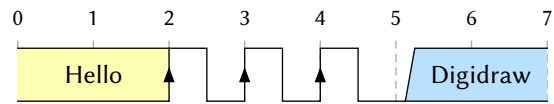
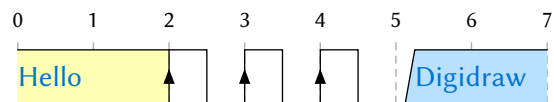
Use debug: `"labels"` to see the bounding boxes of where labels can be placed.

```
#let data = (signal: ((wave: "3.P..5.", data: "Hello
Digidraw")),)

*Default*
#dd.wave(data)

*data-format: `emph`*
#dd.wave(data, data-format: name =>
align(horizon+center, emph(name)))

*data-format: blue text*
#dd.wave(data, data-format: (name) =>
align(horizon,text(blue, name)))
```

Default**data-format: emph****data-format: blue text**

```
Default: data => align(center + horizon, text(
  0.9em,
  bottom-edge: "baseline",
  data,
  ))
```

bus-colors dictionary

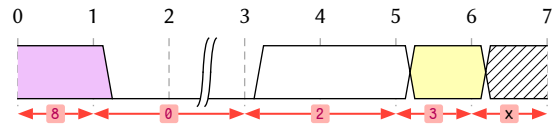
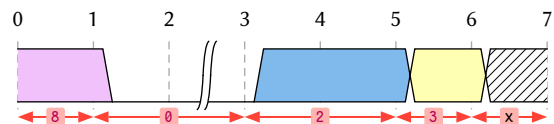
A dictionary containing the bus number/type as the key and fill colors, tilings or gradients as value.

Note When not all bus colors are configured, the remaining ones **will** be set to the default values.

```
#let data = (signal: (
  (wave: "80|2.3x"),)
)
#let bus-colors = ("2": blue.lighten(50%))

#dd.wave(data, debug: "symbols")

*New Bus Colors*
#dd.wave(data, debug: "symbols", bus-colors: bus-
colors)
```

**New Bus Colors**

`digidraw-x-pattern()` is the gray diagonal lines (as seen in the example below). Checkout `-digidraw-x-pattern()` for customization.

Default: (

```
"=": none,
"2": white,
"3": rgb("#ffffb4"),
"4": rgb("#ffe0b9"),
"5": rgb("#b9e0ff"),
"6": rgb("#ccfdfe"),
"7": rgb("#cdfdc5"),
"8": rgb("#f0c1fb"),
"9": rgb("#f8d0ce"),
"x": digidraw-x-pattern(),
)
```

others `cetz`

After the diagram is rendered, additional `cetz` elements can be added on top of it. The parameter accepts a function with following shape

`(info) ⇒ {...}`

where `info` consists of various information useful when placing elements (the values of these properties are given **in `cetz` units**):

`(total-width, total-height, size-ref, symbol-height, wave-origins)`

total-width is equal to the longest wave and can be read from the tick numbers

total-height is equal to the sum of all wave heights and gutters

size-ref (or **symbol-width**) is useful when you want to convert lengths to respective `cetz` units or in reverse.

symbol-height is equal to the symbols' height

wave-origins the actual points used to draw the waves (equal to the top left corner of a wave).

Use debug: `"coordinates"` to find specific points. Additionally you can use `set-origin(...)` to jump to specific points the diagram:

- `"origin"` is the top left corner of the diagram, indicated by the blue circle with a dot inside in the example.
- `"wave.X"` where X is the 0-indexed number of the respective wave. Is equal to the **south west corner** of each wave, also indicated by the red dots in the example.
- `"wave.X.<dir>"` where X is the 0-indexed number of the respective wave and `<dir>` is one of the 8 compass directions (i.e. `"wave.0.north"`).

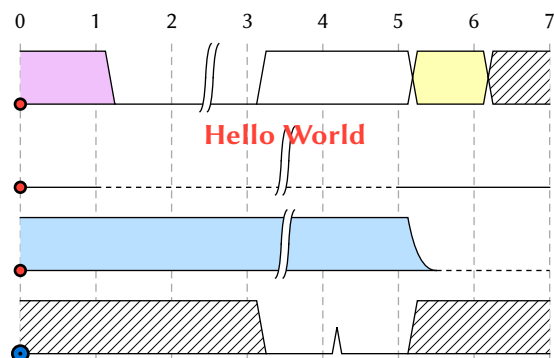
```
#import "@preview/cetz:0.4.2"

#dd.wave((signal: (
  (wave: "80|2.3x"), (wave: "0d.|.l."),
  (wave: "5..|d."), (wave: "xxx00xx"),
)), others: (info) ⇒ {
  import cetz.draw: *

  set-origin("wave.1.north")
  content((0,0),text(red, strong[Hello World]))

  for i in range(4) {
    set-origin("wave." + str(i))
    circle((0,0),radius: 2pt, fill: red)
  }

  set-origin("wave.origin")
  circle((0,0),radius: 3pt, fill: blue)
  circle((0,0),radius: 0.5pt, fill: black, stroke:
0.5pt)
})
```



Default: `none`

tick-layer `int`

The `cetz` layer used to draw the tick lines and numbers. Defaults to `-2` to allow placing elements between ticks and waves using `others` (do note that elements on the same layer as `tick-layer` are placed on top of the ticks).

Default: `-2`

wave-layer `int`

The `cetz` layer used to draw the waves. Defaults to `0` to allow placing elements between ticks and waves using `others`.

Default: `0`

name `string`

This name can be used when referencing anchors from the diagram to attach `cetz` elements to it.

Default: `"wave"`

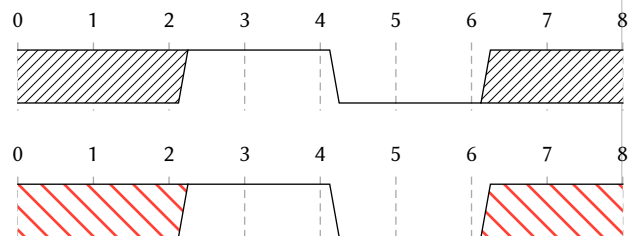
IV.3 digidraw-x-pattern

The default tiling/fill for the `"x"`-bus (but not limited to it). It can be customized with color (`stroke`), sizing (`size`) and direction (`dir`).

```
#let data = (signal: ((wave: "x.1.0.x.")),)

*Default*
#dd.wave(data)

#dd.wave(data,
  bus-colors: (
    "x": dd.digidraw-x-pattern(
      stroke: red,
      size: 3mm,
      dir: ttb
    )
  )
)
```

Default

IV.3.a Parameters

```
digidraw-x-pattern(
  stroke: stroke,
  size: length,
  dir: direction
) → tiling
```

stroke `stroke`

Color of the diagonal lines

Default: `black + 0.3pt`

size **length**

Vertical or horizontal spacing of the lines. The tiling uses a square box, in which one line segment is drawn.

Default: 1.25mm

dir **direction**

Direction of the diagonal lines.

- btt goes from bottom-left to top-right
- ttb goes from top-left to bottom-right

Default: btt

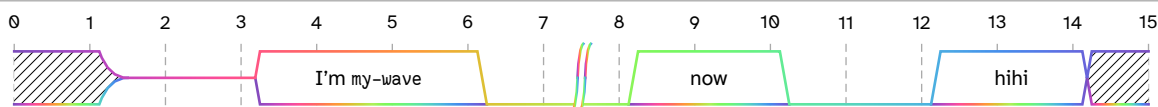
V \ Tips & Tricks

V.1 Pre-Configuring the #wave Function

To preconfigure a function or set a «Global» configuration, you can use the [with](#)-function to pre-apply parameters.

```
#let my-wave = dd.wave.with(stroke: gradient.linear(..color.map.rainbow) + 1pt)

#my-wave(
  (signal: (
    (wave: "xzz2..0|2.0.2.x", data: ([I'm `my-wave`, [now], [hihi],)),)
  )
)
```



V.2 Making Diagrams Referenceable

To make timing diagrams referenceable, enclose it in a #figure element. Additionally, to make labelling (... <a-label>) work, define a new function or «overload» the #wave function:

```
#let wave(..args, caption: none) = figure(
  dd.wave(
    ..args
  ),
  caption: caption,
  supplement: [Diagram],
  kind: "timing-diagrams"
)

#wave(
  (signal: (
    (wave: "x12.udx34z.5.x.", data: "Hello This is Digidraw"),)
  ),
  caption: [Use case of a timing diagram.]
) <dd:example>
```

When a need to reference the @dd:example occurs, `@dd:example` can be used. And for listing the diagrams, the outline target `figure.where(kind: "timing-diagrams")` is to be used:

```
```typst
#outline(target: figure.where(kind: "timing-diagrams"))
```
```

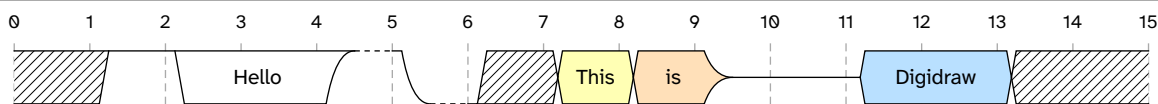
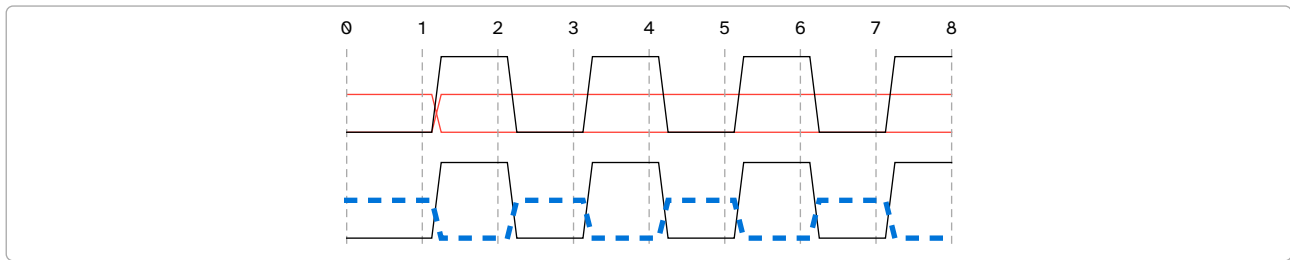


Diagram 1: Use case of a timing diagram.

When a need to reference the Diagram 1 occurs, @dd:example can be used. And for listing the diagrams, the outline target figure.where(kind: "timing-diagrams") is to be used:

```
#outline(target: figure.where(kind: "timing-diagrams"))
```

VI \ Symbol Matrix

The symbol matrix shows all symbol combinations. This is also how the system is debugged (to some degree). If you find any wrong combinations, please report them at <https://codeberg.org/joelvonrotz/typst-digidraw>!

Due to the sheer size of the table, it's rather small. So it's recommended to use a PDF viewer instead of a printed version. Or checkout the matrix as a standalone document over at https://codeberg.org/joelvonrotz/typst-digidraw/src/tag/0.9.4/docs/symbol_matrix.pdf,

| | x | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | p | P | n | N | L | L | h | H | z | u | d |
|---|
| x | |
| 0 | |
| 1 | |
| 2 | |
| 3 | |
| 4 | |
| 5 | |
| 6 | |
| 7 | |
| 8 | |
| 9 | |
| p | |
| P | |
| n | |
| N | |
| L | |
| L | |
| h | |
| H | |
| z | |
| u | |
| d | |
| l | |
| . | |

VII \ Changelog

Version 9.0 – *Initial release of digidraw*

- Implemented the function `#wave` which supports the main parts of the WaveDrom syntax.
- Added various customization settings to the `#wave` function to allow cool styling

Version 9.1

- Typo fixes

Version 9.2

- **Merge-Request by ALVAROPING1**
Expanded `stroke-tick-lines` to support a function with one parameter indicating the tick number. As a return value, either a dictionary (containing the stroke parameters) or a stroke is expected. This allows for styling tick lines at odd or even tick numbers
- Inspired by ALVAROPING1's feature, the tick line system has been expanded to also allow function based visibility and stroke style resetting for specific lines
- fixed some symbol transitions (transition from `z` symbol to `p`, `P`, `n`, `N`, `l`, `L`, `h`, `H`)
- added one new example to show of the new features
- updated manual to be on par with package version 0.9.2

Version 9.3

Highlight

- Internal refactoring of the `#wave` function
 - almost all symbols have now an upper part, a lower part and a body. Upper parts are merged together, leading to one long line, which then are decorated with the lower parts. The body is added behind these parts (see buses).

Added

- Added `edge-overshoot` parameter to `#wave` to allow symbol lines to go over the wave edges (left and right)
- Added `tick-gutter` parameter, that defines the space between the tick numbers and the tick line (including `tick-overshoot`)
- Added `step3` parameter, that controls the end point of the bezier curves (i.e. symbols `"z"` or `"d"`)
- Added `bezier-controlpoint` parameter, that controls the beziers' control point (to make the curve ; might change the name at some point)
- Added `s-spacing`, `s-width` and `s-outside` parameters, that control the shape of the `s`-symbols of a time delay (symbol `"|"`)
- Added `others` parameter, which allows for additional content to be added on top of the diagram. Using `"coordinates"` debug mode helps with finding the correct coordinates.

Changed

- `symbol-width` is now the size reference of the diagram
- Renamed `wave-height` to `symbol-height` and changed allowed value types
- Renamed `stroke-guides` to `guide-stroke` and added support for styling upper and lower guide lines
- Renamed `show-tick-lines` to `show-ticks`
- Renamed `stroke-tick-lines` to `tick-stroke`
- Renamed `inset-1` to `step1`
- Renamed `inset-2` to `step2`

- Split Debugging into four parts: "labels", "symbols", "steps" and "coordinates" (multiple can be activated)
- Updated docs to reflect the changes
- Adapted Examples to new changes

⊖ Removed

- Removed wave-width parameter, because I currently don't have an idea how to implement this with the new system. Plus, the user should be smart enough on how to limit the widths of a diagram by changing the wave strings.
- Removed show-guides parameter
- Removed debug-offset parameter (I might reimplement this at some point)
- Removed size parameter, as symbol-width represents this parameter
- Removed ability to reset configurations by passing **auto** to the respective parameter

Version 9.4

🔗 Highlight

- #subwave makes it possible to place waves inside other waves or in cetz canvases
- #wave.stroke can now also be a function in the shape of (n) ⇒ stroke, where n is equal to the wave index (either use debug: "anchors" to see the index on the right side or)
- Flipped number signs and calculated offsets correctly, so cetz's canvas axis direction do not have to be changed. This makes it easier to place waves inside cetz canvases.

⊕ Added

- Added #wave.name parameter to name diagrams (for anchoring)
- Added debug: "anchors" to see the most important anchor points and wave names.
- Added #subwave to make placing waves inside waves easier. Also works with placing in cetz graphs

⦿ Changed

- how anchors are referenced. Before "waveX" was used, where X represented the respective wave's index number. Now it is in the shape of "<name>.<index>.<direction>"
 - <name> is the diagram's name, defaulting to the value "wave"
 - <index> is similar to the X parameter of the old format
 - <direction> (optional) is the compass direction ("north", "north-west", etc. & "center") to make placing a bit more specific. Not setting any direction, defaults to "south-west"
- Changed cetz canvas to not use any scaling anymore, to make placing waves inside cetz canvases easier
- #wave.stroke supports functions, so for example odd indexed waves can be colored red.
- Updated docs to reflect new changes

Credits

This manual and the other docs use at least one of the fonts below:

- Atkinson Hyperlegible Next – Headings
- Buenard – Used in banner2.typ
- Libertinus Sans – Text
- Fantasque Sans Mono – Code / Raw
- Tabler Icons – Iconography in Callouts

Disclaimer

No AI/LLM generated code or tools were used, as this package was a personal challenge to build a complex Typst package, while still understand how the insides work. It takes time, but nothing runs away.

Additionally, I do not allow nor support AI/LLM contributions to this package. This package is probably already in a database of non-consently crawled data.

Learning brings me joy and I feel bored when using AI/LLMs. It has no value in arts and craft. Support your local artists!



Image source: https://commons.wikimedia.org/wiki/File:Human_made_logo_by_Matt_Kindt.png